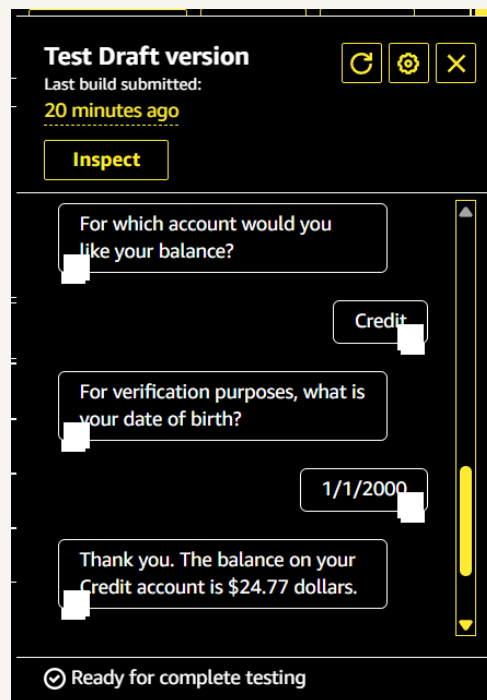




Save User Info with a Lex Chatbot

 Michael K.





Introducing Today's Project!

What is Amazon Lex?

Amazon Lex is a fully managed artificial intelligence (AI) service from AWS that lets you build voice and text conversational interfaces using Natural language.

How I used Amazon Lex in this project

In today's project, I used Amazon Lex to save User Info for sometimes and enable the Bot to use it during the Conversation with the user.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how evolved my Bot looked like after that.

This project took me...

This project took me an hour.



Context Tags

Context tags are used to store and check for specific information across different parts of a conversation. They help save the user from having to repeat certain information

There are two types of context tags: Input and Output Context Tags. Input context tag checks if specific details are already available before an intent activates. For example, FollowupCheckBalance will check if this conversation already has the user's date of birth saved somewhere, so it won't need to ask for that information again. Output context tag tells the chatbot to remember certain details after an intent is finished, so other parts of the conversation can use this stored information later. For example, the account type from CheckBalance could be saved and reused.

I created a context tag called contextCheckBalance in the CheckBalance.

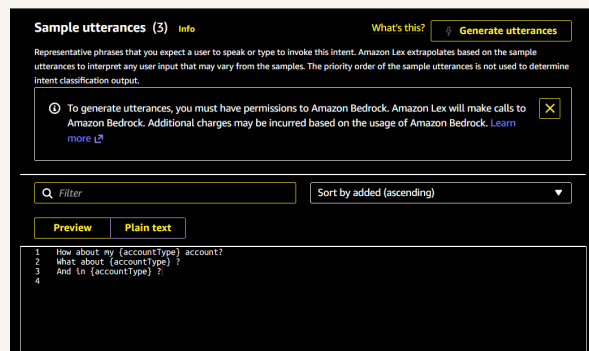
The screenshot shows a dark-themed dialog box titled "Add new context tag" with a close button (X) in the top right corner. Inside the dialog, there is a text input field labeled "Context tag name" containing the text "contextCheckBalance". Below this, there is a section labeled "Expires after" with two input fields: the first contains "5" and the second contains "90", with the text "turns, or" between them and "seconds" at the end. At the bottom right of the dialog are two buttons: "Cancel" and "Add".



FollowUpCheckBalance

I created a new intent called FollowupCheckBalance. The purpose of this intent is to handle follow up questions without asking for the user's birthday again.

This intent is connected to the previous intent I made, CheckBalance, because it uses the value dateOfBirth from the CheckBalance.





Input Context Tag

I created an input context, contextCheckBalance, that will check if the Bot already have the information he needs to repond to the user intent before doing so it doesnt ask the information twice. It takes that information from the Output Context.

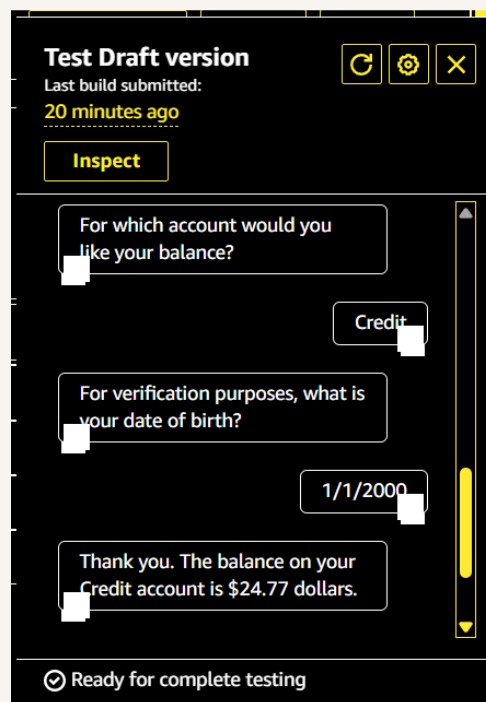
The screenshot shows a dark-themed dialog box titled "▼ Default values - optional". Inside, there is a text input field containing the value "#contextCheckBalance.dateOfBirth" with a yellow 'X' icon to its right. Below this field is a horizontal line. Underneath the line, a small instruction reads: "Provide a default value, #value for a context value, or [variable] for session variable." Below the instruction is another text input field containing the placeholder text "San Diego, #ContextTag.SlotName, [SessionAttributeName]". To the right of this field is a yellow button labeled "Add default value". At the bottom right of the dialog are two buttons: a yellow "Cancel" button and a grey "Update slot" button.



The final result!

To see the context tags and followup intent in action, I used one of the Utterances i defined in the FollowupCheckBalance Intent.

If I had gone straight to trying to trigger FollowUpCheckBalance without setting up any context the bot would not acknowledge my intention and would respond with a message asking about how it can help me.





Managing context expiry

An extension for this project is to manage contextCheckBalance's context expiry, which means how long the bot will remember information for. By default, expiry is 5 turns or 90 seconds.

I updated my bot's context expiry to a short memory (short window). What this means for my end users is that their sensitive informations will not be remembered by the bot for a long time, thus the assurance that attackers will have less chance to access these informations.

A long context expiry window would be helpful when one expect the user to have long conversations and run through multiple intents in the same session. Ideally they wouldn't have to share the same information again and again! . A shorter context expiry window would be helpful when the information is sensitive and I'd want to protect the user from attackers getting access to chat windows with lots of active contexts.



Edit context ✕

Context tag name

Expires after
 turns, or seconds

Cancel Save